

University Participation System - System Manual

Token-based surveys & event registration with email-verified participants

University Participation System

2026

- 1. Overview
- 2. What it solves
- 3. Key concepts
- 4. Architecture
- 5. Data model
- 6. End-to-end flow
 - 6.1 Administrator flow
 - 6.2 Student flow
 - 6.3 Request flow (technical)
- 7. Administrator guide
 - 7.1 Create / select an event
 - 7.2 Build the form
 - 7.3 Upload participants
 - 7.4 Distribute links
 - 7.5 Monitor & export
- 8. Student guide
- 9. Email (OTP) setup
- 10. Deployment & configuration
 - 10.1 Environment variables
 - 10.2 Render
 - 10.3 Docker / VPS
- 11. API reference
- 12. Security model
- 13. CSV / file formats & column detection
- 14. Troubleshooting
- 15. Local development

1. Overview

The **University Participation System** is a secure, general-purpose web application for running **event registrations and surveys** where only **eligible, registered people** can take part. Each eligible person

receives a **personal link** (a token). They verify ownership of their **registered email** with a **one-time code (OTP)** and then complete a **form** that the administrator designs. Every token can be used **once**.

It replaces “open Google Forms” links, which anyone with a valid-looking email can fill in — including alumni or outsiders. Here, **access is granted only to the people whose records (including email) the administrator uploads**.

Live deployment: Render (web service + Postgres), email via Brevo HTTP API.

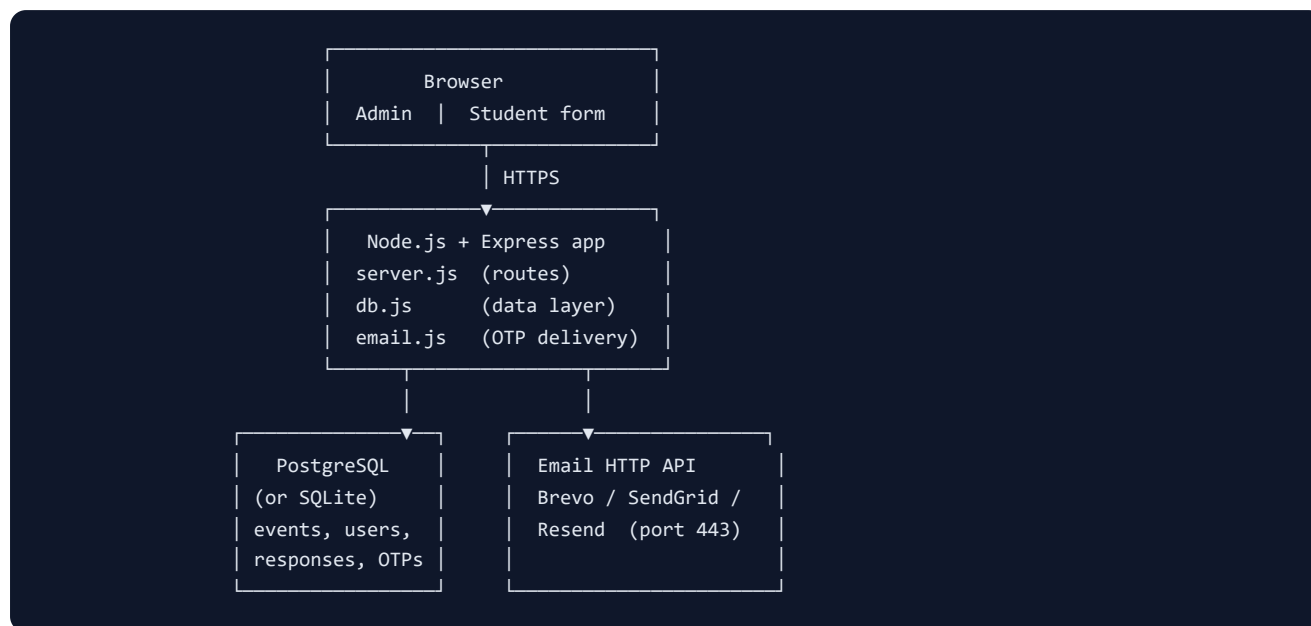
2. What it solves

Risk with open forms	How this system fixes it
Anyone with the link can submit	No public form link; only personal token links
Alumni / outsiders with valid email	Only uploaded emails can verify
One person submitting many times	Token is single-use and OTP is single-use
Email might be spoofed	Only the exact registered email receives the code
Data hard to analyse	Live charts + CSV/PDF exports
Rebuilding forms for each event	Create / duplicate events with custom fields

3. Key concepts

- **Event** — a form/registration campaign (e.g. *Dean's Cup 2026*). Each event has its own **form fields**, **participants**, and **responses**.
 - **Participant** — one eligible person in an event. Stored as `reg_number`, `full_name`, `email`.
 - **Token** — a random 32-character id generated per participant. It forms the personal link: `.../s?token=XXXX`. **Single-use**.
 - **Form fields** — the questions shown to the student, defined per event (text, choice, multi-choice, etc.).
 - **OTP** — a 6-digit one-time code emailed to the participant's registered email. Expires in **10 minutes**, **single-use**.
 - **Admin key** — a shared secret required for all admin actions.
-

4. Architecture



- **Frontend:** static HTML pages served by the app (/ , /admin.html , /s), styled with Tailwind (CDN), charts with Chart.js, PDF via jsPDF, spreadsheet reading via SheetJS.
- **Backend:** `server.js` (Express) exposes the API and renders the survey page.
- **Data:** `db.js` supports **PostgreSQL** (production) and **SQLite** (local/zero-config). Schema is created automatically on start.
- **Email:** `email.js` sends OTPs through an **HTTP email API** (needed because most PaaS hosts block outbound SMTP ports).

5. Data model

events — one row per form/event

column	notes
id	primary key
slug	unique short id (e.g. <code>deans-cup-2026</code>)
title	display name
description	optional
fields	JSON array — the form definition
created_at	timestamp

participants — eligible people (one token each)

column	notes
token	primary key (used in the personal link)
reg_number	registration / voter number (unique per event)
full_name	name
email	the only email allowed to verify
used	0/1 — set to 1 after submission
event_id	which event
extra	JSON of the full uploaded row
created_at	timestamp

responses — one row per submission

column	notes
token	participant token
reg_number, full_name	copied
gender, phone, year_of_study, sports, football_position, other_position	common legacy columns
data	JSON of all answers (flexible fields)
event_id	which event
submitted_at	timestamp

otps — one-time codes

column	notes
token, email	who it belongs to
otp	6-digit code
expires_at	10 minutes after creation
verified	0/1 (single-use)

6. End-to-end flow

6.1 Administrator flow

1. Open admin page (admin key required)
|
2. Create or select an event
|
3. Build the form fields
(visual builder: add fields, types, options)
|
4. Upload the eligible-participants file
(CSV / TSV / Excel)
→ columns auto-detected: reg/voter no, name, email
→ optional: also use these columns as the form fields
|
5. System generates one personal token per participant
|
6. Export the tokens CSV / send each student their link
|
7. Monitor responses, charts, export CSV / PDF
|
8. (Optional) Duplicate the event for the next campaign

6.2 Student flow

```
Personal link .../s?token=XXXX
|
▼
[Token checked]  invalid? → "Invalid link"
                  already used? → "Already submitted"
|
▼
Step 1: email shown (locked) → "Send verification code"
|
▼
Email inbox ← 6-digit OTP (10-min, single-use)
|
▼
Step 2: enter code → verified
|
▼
Step 3: fill the event form → Submit
|
▼
Response saved · token + code become unusable
```

6.3 Request flow (technical)



7. Administrator guide

Open the admin page. The **key** can be embedded in the URL so you don't retype it:

```
https://<your-app>/admin.html?key=<ADMIN_KEY>
```

A ✓ **connected** / ✗ **invalid key** indicator shows top-right. A red banner appears if the key is wrong.

7.1 Create / select an event

Use the **event dropdown** to switch events. + **New Event** creates one (slug + title). **Duplicate Form** copies the selected event's form fields (optionally its participants with fresh tokens). **Delete Event** removes the event with all its participants and responses.

7.2 Build the form

In **2. Build the form students fill**: - + **Add field** → each row: Name (key), Label, Type, Required, Options. - Types: **text**, **textarea**, **number**, **email**, **tel**, **date**, **select**, **radio**, **multiselect**. - For dropdowns/checkboxes put choices in **Options**, separated by **|** (e.g. **Male|Female**). - **Load Dean's Cup default** restores the built-in 8-field form. - **Upload fields file (CSV/JSON)** loads a template; **Download template CSV**. - **Save form for this event** persists it. **Preview** shows how students see it.

7.3 Upload participants

In **1. Upload participants**: - **Choose file** (CSV / TSV / Excel) then **Upload & Generate Tokens**. - Columns are auto-detected: registration/voter number, name, email. - Tick **“Also replace the form fields with these CSV columns”** if you want the form questions to mirror your file’s columns. - A warning appears if **no email column** is found (those people can’t verify).

Required logical columns: a **registration/voter number**, a **name**, and an **email**. Extra columns are stored too.

7.4 Distribute links

In **3. Participants & Personal Links**, each row shows the personal link and an **Email** column. **Export tokens CSV** to get every link. Send each person **only their own** link.

7.5 Monitor & export

In **4. Responses**: the table shows all answers. Buttons: - **Download responses CSV** — one column per form field. - **Download PDF** — a landscape PDF table of all responses. Charts below are generated automatically from each **select** / **radio** / **multiselect** field, plus a submissions-over-time chart.

8. Student guide

1. Open your **personal link** (`.../s?token=...`).
2. Your **registered email** is shown (you cannot change it). Click **Send verification code**.
3. Check your inbox (and spam) for a **6-digit code**; enter it and click **Verify**.
4. Fill the form and click **Submit**. You’ll see a confirmation.
5. That’s it — the link cannot be used again.

The code expires after **10 minutes**. You may click *Send verification code* again to get a new one.

9. Email (OTP) setup

Real email requires an **HTTP email API** because Render (and many hosts) **block outbound SMTP ports (25/465/587)**. The app supports **Brevo** (recommended free tier: 300/day), **SendGrid**, **Resend**, plus plain **SMTP** for hosts that allow it.

Brevo quick start 1. Create a free Brevo account. 2. **Senders** → add and **verify** the sender email. 3. **SMTP & API** → **API Keys** → create an API key (starts with `xkeysib-`). 4. **Security** → **Authorised IPs** → **disable** the IP restriction (Render's IPs change). 5. Set the environment variables (see §10).

If email is **not** configured, the app runs in **demo mode** and shows the code on screen (never use demo mode for real events).

10. Deployment & configuration

10.1 Environment variables

variable	purpose
<code>PORT</code>	server port (host-provided on Render)
<code>ADMIN_KEY</code>	secret for all admin actions — set a strong value
<code>DATABASE_URL</code>	PostgreSQL URL (recommended). If unset → SQLite
<code>SQLITE_FILE</code>	SQLite path (when no <code>DATABASE_URL</code>)
<code>EMAIL_PROVIDER</code>	<code>brevo</code> / <code>sendgrid</code> / <code>resend</code> / <code>smtp</code>
<code>BREVO_API_KEY</code>	Brevo API key (<code>xkeysib-...</code>)
<code>SENDGRID_API_KEY</code> / <code>RESEND_API_KEY</code>	alternative providers
<code>EMAIL_FROM</code>	sender, e.g. <code>Dean's Cup <no-reply@domain></code>
<code>APP_NAME</code>	name shown in emails
<code>SMTP_HOST/PORT/SECURE/USER/PASS/FROM</code>	only where SMTP ports are open

10.2 Render

- The repo contains `render.yaml`. Create a **Blueprint** from the repo, or a **Web Service** with build `npm install` and start `node --experimental-sqlite server.js`.
- Add a **PostgreSQL** instance and set `DATABASE_URL` to its **internal** URL so data persists across deploys.
- Set `ADMIN_KEY`, `EMAIL_PROVIDER=brevo`, `BREVO_API_KEY`, `EMAIL_FROM`.

10.3 Docker / VPS

```
docker compose up -d
```

Uses the included `Dockerfile` and `docker-compose.yml` (SQLite on a volume, or Postgres if you set `DATABASE_URL`).

11. API reference

Admin endpoints require the key via header `X-Admin-Key: <key>` or query `?key=` .

Method & path	Auth	Description
GET /	–	Home page listing events
GET /api/events	–	List events
GET /api/events/:slug/fields	–	Form fields for an event
POST /api/admin/events	✓	Create event {slug,title,description}
DELETE /api/admin/events/:slug	✓	Delete event (+ participants/responses)
POST /api/admin/events/:slug/duplicate	✓	Duplicate {newSlug,newTitle,copyParticipants}
GET /api/admin/fields?event=	✓	Get form fields
POST /api/admin/fields	✓	Save fields {event, fields:[...]}
GET /api/admin/fields/sample.csv	✓	Download a fields template
POST /api/admin/import?event=	✓	Import {participants: [{reg_number,full_name,email,...}]}
GET /api/admin/participants?event=	✓	List participants (with tokens)
DELETE /api/admin/participants/:token	✓	Delete a participant (+ response)
GET /api/admin/responses?event=	✓	List responses
GET /api/admin/stats?event=	✓	Aggregated chart data
GET /api/admin/export?event=	✓	Download responses CSV
POST /api/auth/request-otp	–	{token,email} → send code
POST /api/auth/verify-otp	–	{token,email,otp} → verify
GET /s?token=	–	Student form
POST /api/submit	–	{token,email,answers} → save response

12. Security model

- **No public form.** Entry requires a token that only the admin can produce.
- **Allow-list of emails.** A code is sent **only** to the email uploaded for that participant; other emails are rejected.

- **Email verification.** Proves the person controls the registered mailbox.
- **Single-use.** Each token and each OTP code works once.
- **Admin key.** Protects all management actions.
- **HTTPS** on Render; secrets stored as environment variables, not in code.
- **What it is not:** a password vault. The system **never** asks for an email password — only a one-time code.

Recommended hardening for production: use a long random `ADMIN_KEY`, keep the email API key secret, and (optionally) add rate-limiting on OTP requests.

13. CSV / file formats & column detection

Accepts **CSV**, **TSV**, **semicolon CSV**, and **Excel (.xlsx/.xls)**.

Column headers are matched flexibly:

Logical field	Recognised headers (examples)
Registration / voter number	<code>reg</code> , <code>reg no</code> , <code>registration number</code> , <code>matric</code> , <code>voter number</code> , <code>voter no</code> , <code>id</code> , <code>number</code> , <code>no</code> , <code>code</code>
Name	<code>full name</code> , <code>name</code> , <code>voter</code> , <code>student</code>
Email	<code>email</code> , <code>e-mail</code> , <code>mail</code>

Example participant file:

```
Voter Number,Voter,Email
262101010943,ELAJU AARON,elaju@soroti.ac.ug
262101030491,OPOLOT AARON,opolot@soroti.ac.ug
```

Form-field template (for the **Upload fields file** option):

```
name,label,type,required,options
full_name,Full Name,text,yes,
gender,Gender,select,yes,Male|Female
sports,Sports,multiselect,yes,Football|Chess|Swimming
comments,Comments,textarea,no,
```

14. Troubleshooting

Symptom	Cause / fix
"Invalid admin key" banner	Wrong key. Open <code>admin.html?key=<ADMIN_KEY></code> or type the key and Save .
Upload does nothing	You didn't pick a file, or the key is wrong. Watch the message under the button.
"No email column detected"	Your file has no email column — students can't verify. Add an email column.
Code not received	Check spam; confirm <code>EMAIL_FROM</code> is a verified sender; Brevo: disable IP restriction; check server logs.
"This email is not registered for this link"	The typed email $\neq$ the uploaded email. Use the registered one.
"No email is registered for this link"	That participant row has no email. Re-import with emails.
"Already submitted"	The token was used. Issue a new link/participant.
Data disappears after deploy	No <code>DATABASE_URL</code> set (SQLite on ephemeral storage). Attach Postgres.
Email works locally but not on Render	SMTP ports are blocked — use an HTTP email API (Brevo/SendGrid/Resend).

15. Local development

```
npm install
npm start      # http://localhost:3002 (admin.html?key=admin123)
```

Without email variables the app runs in **demo mode** (code shown on screen). Set the email variables in a `.env` file (see `.env.example`) to send real mail.

End of manual.